1. 말로 등록하고 나중에 알림받기

- “빨간 컵이 보이면 알려줘”라고 요청한다.
- 카메라를 계속 보고 있지 않아도 물건이 확인되면 위치를 알려 준다.
- 현재 장면에 답하는 대신 사용자의 조건을 기억하고 기다리는 도우미다.
- 한 번에 물건 하나를 찾으며 물건 목록은 YOLO에서 자동으로 읽는다. 색상은 8종을 지원한다.

2. 모델의 역할

- Whisper: 녹음한 요청을 텍스트로 바꾼다. 무음이면 등록하지 않는다.
- Gemma: 시작할 때 한 번 불러오고 요청마다 물건과 색상 조건을 해석한다.
- YOLO가 종류를 찾고, OpenCV는 중앙 영역의 HSV 색상 비율을 확인한다.
- Piper: 등록 확인과 발견 위치를 음성으로 알려 준다.

3. 발견했을 때만 한 번

- 종류와 색상이 세 번 연속 맞아야 발견으로 판단한다.
- 중간에 탐지되지 않으면 확인 횟수를 0으로 돌린다.
- “찾으시던 빨간 컵이 화면 왼쪽에 보입니다”처럼 위치를 안내한다.
- 알린 뒤 찾기를 종료해 같은 물건을 계속 읽지 않는다.

4. Nano에서도 설명하기 쉬운 구조

- main.py의 메인 스레드는 카메라, 작업 스레드 하나는 음성 처리를 맡는다.
- 조건은 물건과 색상만 기억한다. 색상을 생략하면 종류만 확인한다.
- 영상은 320×240, 10 FPS로 전달받고 YOLO 입력도 320으로 줄였다.
- Gemma를 재사용해 로딩 시간을 줄인다. 대신 YOLO와 함께 RAM을 사용한다.

5. 장비 시연 순서

- v를 누르고 “빨간 컵이 보이면 알려줘”라고 말한다.
- 파란 컵에는 알리지 않는지 확인한 뒤 빨간 컵을 왼쪽으로 가져온다.
- 연속 확인 후 위치 알림을 듣고, 그대로 두어도 반복하지 않는지 확인한다.
- h는 지원 목록, c는 취소, q는 종료다. 요청 처리 중에도 미리보기를 유지한다.

6. 창의성과 한계

- 자연어 요청을 나중에 실행할 관찰 조건으로 연결한 점이 핵심이다.
- 색상 비율 검사는 조명과 배경의 영향을 받는다. 특정 주인은 구분하지 못한다.
- 낮은 탐지 빈도 때문에 잠깐 보이는 물건을 놓칠 수 있다.
- 실제 Nano 시연과 응답 시간, 메모리 측정은 별도로 확인한다.

코드와 문서 정리에 생성형 AI의 도움을 사용했다. 실행하지 않은 성능 결과는 작성하지 않았다.